

JavaScript Generators

[Back to JS page](#)

Table of Contents

- [JavaScript Generators](#)
 - [Generator Functions](#)
 - [Example 1](#)
 - [The yield Keyword](#)
 - [Example 2](#)
 - [Generator Object Methods](#)
 - [Example 3](#)
 - [Custom Iterators with Generators](#)
 - [Example 4](#)
 - [Document](#)
 - [Reference](#)

Generator Functions

A generator function is a special function that can pause and resume execution. It is defined with `function*`:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Generator Functions</h4>
<p id="demo"></p>

<script>
// Generator function defined with function*
function* myGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

// Calling a generator returns a generator object
const gen = myGenerator();

let text = "Generator function created<br><br>";

// Iterate over values
for (let value of gen) {
  text += "Yielded: " + value + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

The yield Keyword

The `yield` keyword pauses the generator and returns a value. The generator resumes when `next()` is called:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The yield Keyword</h4>
<p id="demo"></p>

<script>
function* fruitGenerator() {
  yield "Apple";
  yield "Banana";
  yield "Orange";
}

const gen = fruitGenerator();

let text = "Using yield with next():<br><br>";
text += gen.next().value + "<br>";
text += gen.next().value + "<br>";
text += gen.next().value + "<br>";
text += "done: " + gen.next().done;

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Generator Object Methods

Generator objects have methods like `next()` and `return()` :

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Generator Object Methods</h4>
<p id="demo"></p>

<script>
function* counter() {
  let i = 0;
  while (true) {
    yield i++;
  }
}

const gen = counter();

let text = "Counter generator:<br><br>";
text += "next(): " + gen.next().value + "<br>";
text += "next(): " + gen.next().value + "<br>";
text += "next(): " + gen.next().value + "<br><br>";

// Other generator methods
text += "typeof gen.next: " + typeof gen.next + "<br>";
text += "typeof gen.return: " + typeof gen.return + "<br>";

// return() stops the generator
gen.return("stopped");
text += "After return(): " + gen.next().done + " (done)";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>

```

Example 3

Result [View Example](#)

Custom Iterators with Generators

Generators make custom iterators easy to create:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Generators</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Custom Iterators with Generators</h4>
<p id="demo"></p>

<script>
// Create a custom iterable using a generator
const myIterable = {
  *[Symbol.iterator]() {
    yield "One";
    yield "Two";
    yield "Three";
  }
};

let text = "Custom iterable with generator:<br><br>";
for (let value of myIterable) {
  text += value + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Generators](#)